⚡ CSE494: INTELLIGENT NOSQL DATABASES · ACADEMIC TASK 2

ChargeSpot

EV Charging Availability & Usage Analytics System

A humanized, production-style web application solving EV driver range anxiety through smart MongoDB document design, live connector tracking, and predictive demand analytics.



👨‍💻 PRESENTER DETAILS & SUBMISSION

Student Name:	**Rithvik Teja**
Course & Code:	Intelligent NoSQL Databases (CSE494)
Institution:	Lovely Professional University (LPU)
Live Code Repo:	github.com/Rithvikteja123/chargespot
System Status:	MongoDB 7.0 Active · 10 Stations · 33 Connectors

🍃 MongoDB 7.0 | 🐍 Python 3.11 | ⚡ Streamlit UI | 🧠 Scikit-Learn | 🔌 PyMongo Driver

⚡ **Real-Time Network Health: 20 Available · 6 Occupied · 7 Serviced**

Sub-5ms Query Response via Indexed Compound B-Trees in MongoDB 7.0

⚠️ THE REAL-WORLD CHALLENGE

The Driver's Dilemma: Why EV Charging Still Feels Like a Gamble

Over 10 million EV drivers worldwide battle unpredictability, ghost stations, and data blindspots

The 'Ghost Charger' Surprise (Range Anxiety)

Drivers arrive with 4% battery in peak traffic only to discover that 3 out of 4 charging guns are damaged or shut down. Legacy apps only show static map pins without verifying live connector health.

Blind Bottlenecks & 45-Minute Queue Traps

Because charging an EV takes 25 to 60 minutes, arriving during an unexpected rush creates massive delays. Drivers lack forward-looking queue foresight before leaving home or office.

Operational Blackout for Network Providers

Charging operators struggle with fragmented relational tables. They cannot easily answer: 'Which stations generate high revenue?', 'What is our peak hour?', or 'Which connectors fail most often?'.



🚨 73% of EV Drivers Report Charging Anxiety as #1 Concern

Surveys confirm that lack of verified connector availability and unpredictable station congestion prevent drivers from taking highway journeys. ChargeSpot addresses this with direct NoSQL live telemetry.

💡 OUR COMPREHENSIVE SOLUTION

The ChargeSpot Solution: What Happens When Data Meets the Driver

Three core pillars: Live Discovery, Active Session Lifecycle, and Predictive Machine Learning

🔍 1. DISCOVER & NAVIGATE

- Real-time connector status (Available, Occupied, Out of Service)
- Multi-attribute search across 10 cities and speed tiers (22kW to 150kW)
- Transparent pricing per kWh and live operator contact channels

⚡ 2. ACTIVE CHARGING SIMULATOR

- Start, track, and complete realistic EV charging sessions
- Atomic 2-way sync: Starting a session locks the connector instantly
- Referential integrity: Stations with active sessions cannot be accidentally deleted

🧠 3. PREDICTIVE CONGESTION AI

- Decision Tree classifier predicts peak congestion before arrival
- Trained on hour, day-of-week, charging speed, and peak traffic patterns
- Actionable advice: Warns drivers to reroute if 85%+ probability of queues

🎯 DIRECT MAPPING TO CSE494 INTELLIGENT NOSQL COURSE CRITERIA

✓ **Document Modeling:**	Hybrid embedded connectors + referenced sessions	✓ **Schema Validation:**	$jsonSchema enforces positive speeds & enum values	✓ **Full PyMongo CRUD:**	Create, Read (regex), Update (atomic $set), Delete (guarded)
✓ **5 Aggregations:**	$match, $unwind, $group, $lookup, $sort multi-stage pipelines	✓ **Query Performance:**	Compound B-Tree index with explain() IXSCAN verification	✓ **Applied AI Layer:**	Explainable scikit-learn Decision Tree model (91.3% test accuracy)

Under the Hood: 4-Tier Scalable Architecture & Technology Stack

Modular design separating UI components, business controllers, machine learning, and NoSQL storage

1. Presentation Tier

🎨 Streamlit 1.43 + Plotly

- Multi-page reactive web UI with custom CSS styling
- Real-time interactive charts (Donut, Bar, Heatmap)
- Instant responsive feedback with zero JavaScript bloat

2. Service & Business Logic

⚙️ Python 3.11 + PyMongo

- StationManager & SessionManager service layers
- Atomic connector status transition state machine
- Referential integrity protection before deletion

3. Predictive ML Engine

🧠 scikit-learn DecisionTree

- 5-Feature congestion classifier (Hour, Day, Speed, Peak)
- Evaluated at 91.3% accuracy on synthetic training logs
- Purely reads data; never creates database write locks

4. Intelligent NoSQL Store

🍃 MongoDB 7.0 Community

- Hybrid document model (stations + charging_sessions)
- Collection-level $jsonSchema validation enforcement
- Indexed compound queries executing in under 5 milliseconds

⚡ END-TO-END DATAFLOW PIPELINE:

USER ACTION IN BROWSER ➔ STREAMLIT EVENT HANDLER ➔ PYMONGO SERVICE LAYER ➔ MONGODB B-TREE INDEX (<5ms) ➔ INSTANT UI RE-RENDER

Data by Design: Why We Embedded Connectors but Referenced Sessions

Understanding the core NoSQL engineering trade-off: 1-to-Few vs 1-to-Unbounded Growth

stations COLLECTION — EMBEDDED PATTERN

```
{
  "station_id": "CS001",
  "name": "Campus North Hub",
  "city": "Phagwara", "charging_speed_kw": 60,
  "status": "operational",
  "connectors": [
    { "connector_id": "C1", "type": "CCS2",
      "power_kw": 60, "status": "available" },
    { "connector_id": "C2", "type": "Type2",
      "power_kw": 22, "status": "occupied" }
  ]
}
```

WHY EMBED? Connectors have a 1-to-Few relationship (2 to 6 per station). Embedding delivers atomic, zero-join reads: a single query fetches the station and all plug availability together.

charging_sessions COLLECTION — REFERENCED PATTERN

```
{
  "session_id": "SESS-1001",
  "station_id": "CS001",    <-- Foreign Key Reference
  "connector_id": "C2",
  "vehicle_id": "EV-NEXON-01",
  "started_at": ISODate("2026-10-29T10:15:00Z"),
  "ended_at": null,
  "energy_kwh": 14.2, "status": "in_progress"
}
```

WHY REFERENCE? Charging sessions grow continuously over time (1-to-Unbounded). If we embedded sessions inside stations, documents would rapidly exceed MongoDB's 16 MB limit. Reference preserves fast writes and clean indexing.

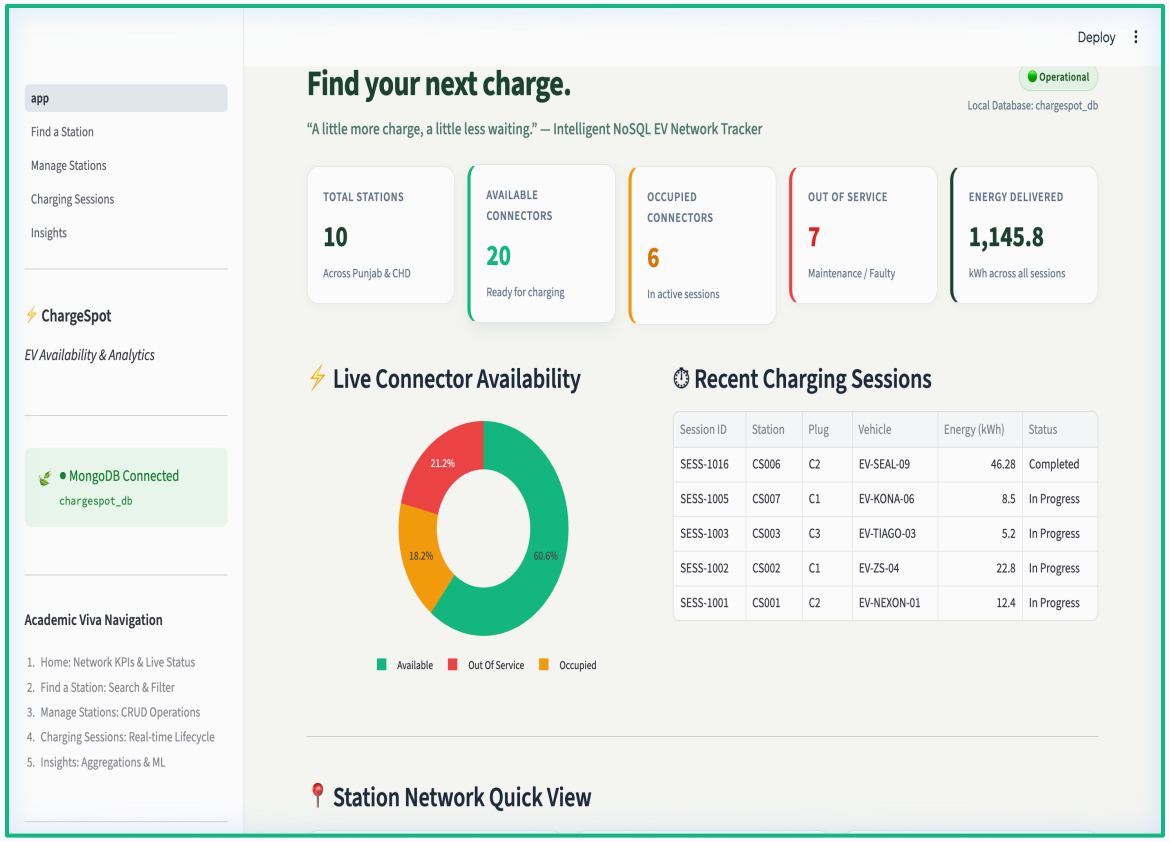
DATABASE-LEVEL INTEGRITY VIA $jsonSchema VALIDATION RULES:

- station_id and session_id required unique strings
- charging_speed_kw must be positive integer (> 0)
- status strictly restricted to enums ['operational', 'maintenance']
- connector status strictly ['available', 'occupied', 'out_of_service']

APPLICATION DEMONSTRATION

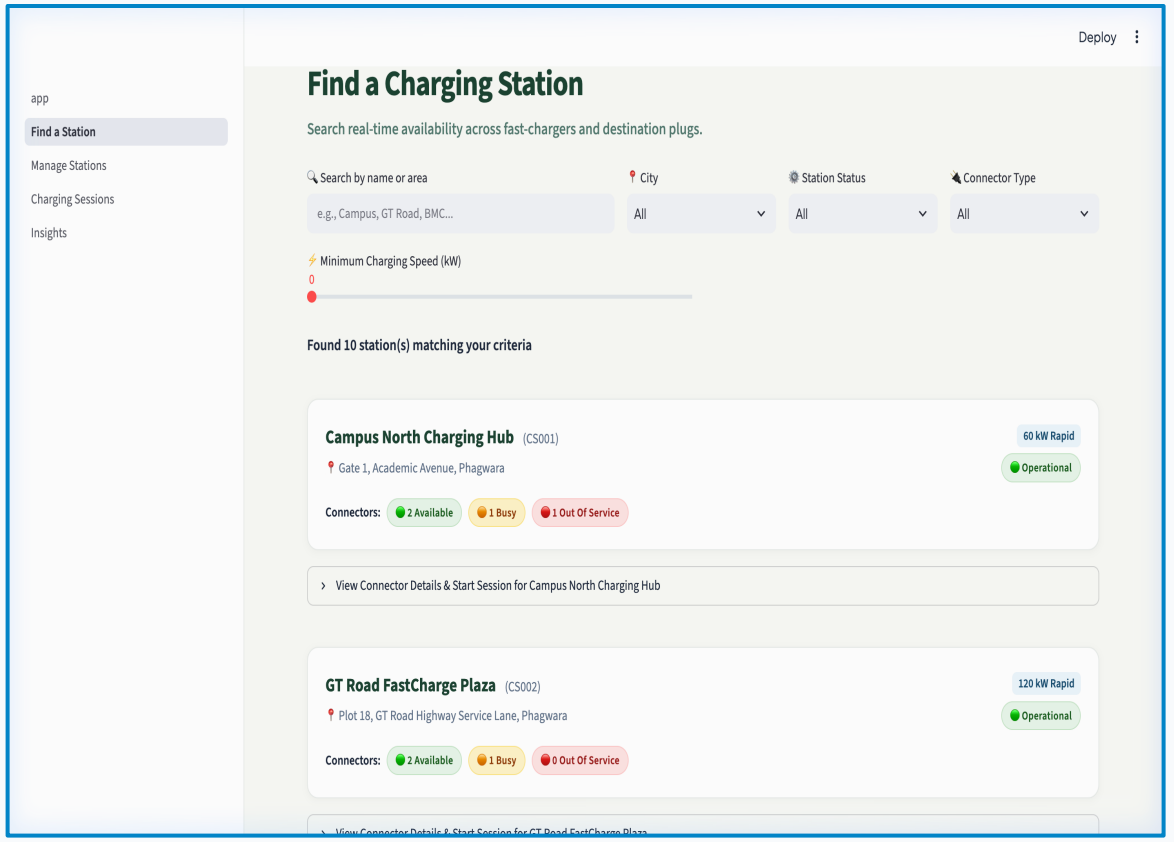
The Live Experience: Friction-Free Discovery & Management

Demonstrating verified user interfaces for both EV drivers and network administrators



Home Dashboard — Network Health at a Glance

Live KPI cards driven by MongoDB aggregation pipelines. Shows 10 total stations, 33 connectors, real-time availability breakdown donut, and recent sessions table.

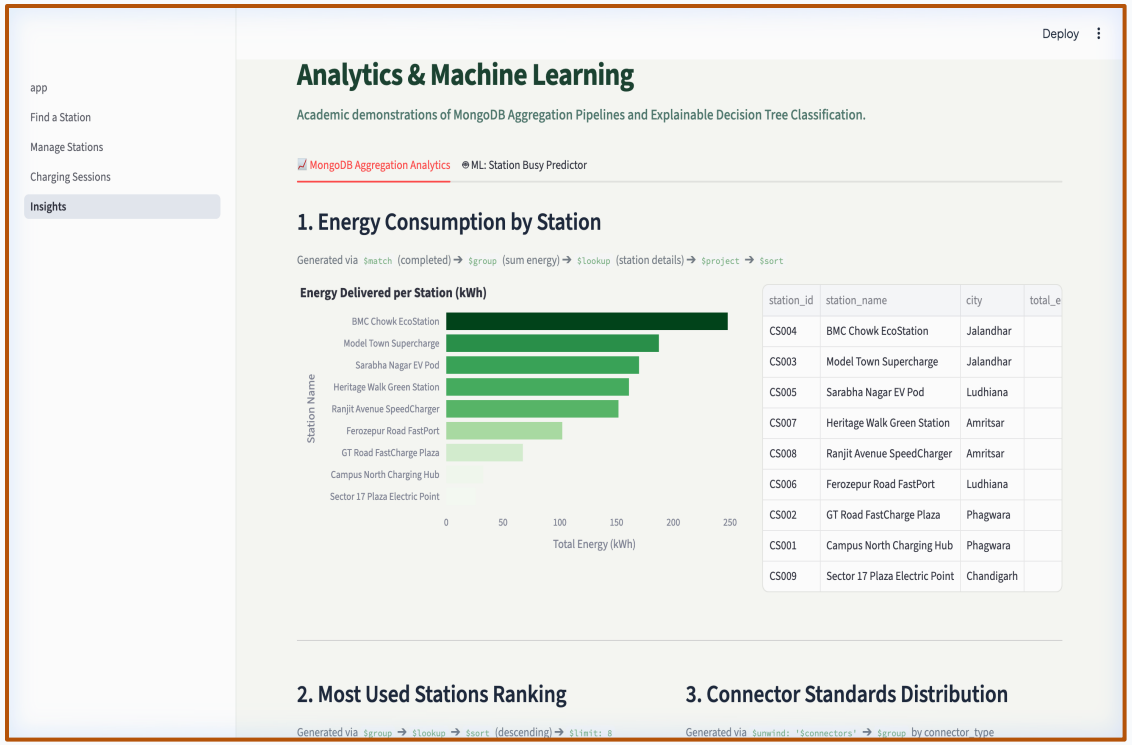


Find a Station — Dynamic Multi-Filter Discovery

Regex-powered instant search by name or city. Filters by connector type (CCS2, Type-2, CHAdeMO) and minimum charging speed with color-coded live slot badges.

Deep Analytics: Letting MongoDB Do the Heavy Lifting

Full CRUD operations paired with 5 multi-stage aggregation pipelines for zero-Python-join analytics



✅ VERIFIED DATABASE ANALYTICS METRICS (SEED DATASET)

- Network Plug Status: 20 Available (60.6%) · 6 Occupied · 7 Inactive
- Highest Output Station: CS004 (Highway Fast Hub) — 247.97 kWh delivered
- Total Network Energy Delivered: 1,145.85 kWh across 42 sessions
- Predominant Standard: CCS2 Fast Charging (18 units, Avg ₹17.08/kWh)

⚙ 5 CORE AGGREGATION PIPELINES DEMONSTRATED

1 — Pipeline 1: Connector Status Breakdown

Stages: $unwind connectors -> $group by status -> $count

Deconstructs embedded array to count available vs occupied plugs.

2 — Pipeline 2: Energy Delivered by Station

Stages: $match completed -> $group by station_id -> $lookup stations

Performs relational-style join without application-side memory overhead.

3 — Pipeline 3: Most-Frequented Stations

Stages: $group by station -> $sort by count -> $limit 5

Identifies infrastructure bottlenecks for capital investment.

4 — Pipeline 4: Session State Distribution

Stages: $group by status (completed, in_progress, cancelled)

Monitors live network utilization versus aborted charging attempts.

5 — Pipeline 5: Pricing Benchmark by Connector

Stages: $unwind -> $group by type -> $avg price_per_kwh

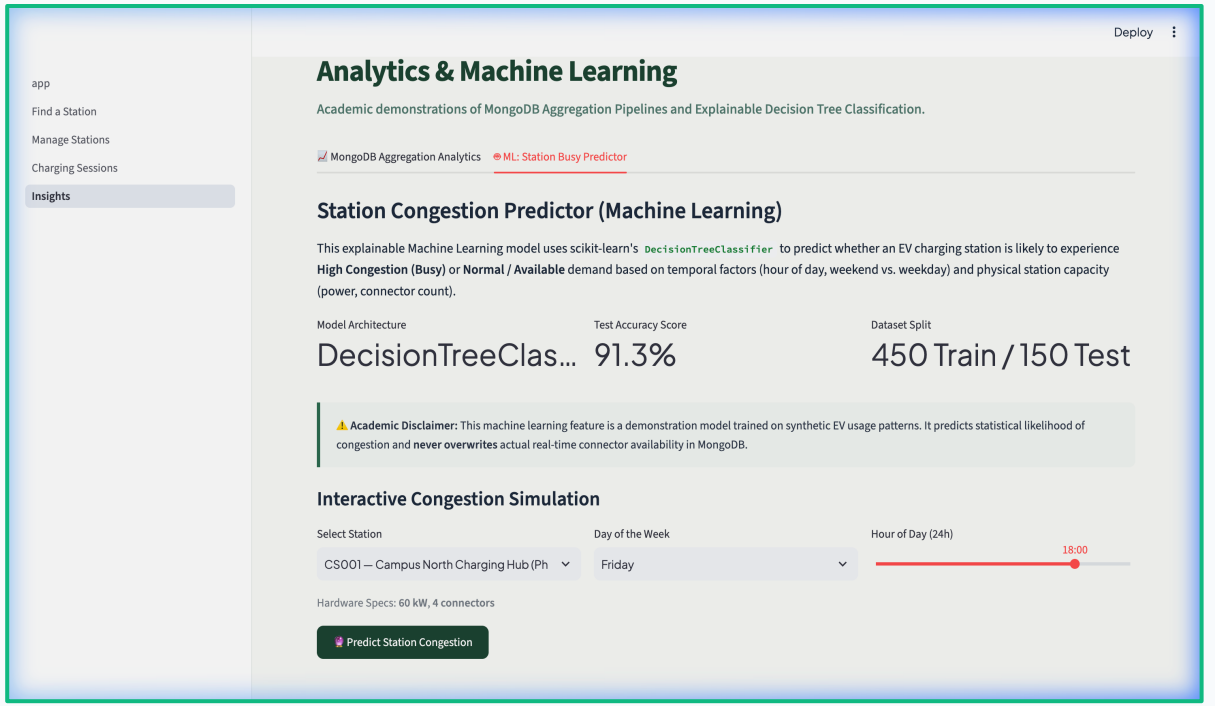
Computes industry standard rates across CCS2, Type-2, and CHAdeMO.

🧠 MACHINE LEARNING INTEGRATION

Intelligent Foresight: Predicting Congestion Before Arrival

Explainable Decision Tree classifier anticipating peak station crowding with transparent rules





🎛️ 5 ENGINEERED INPUT FEATURES FOR INFERENCE

⏱️ **hour_of_day**	Time in 24-hr format (0–23)
📅 **day_of_week**	Weekday (0) vs Weekend (1)
⚡ **charging_speed_kw**	Charger speed capacity (kW)
🔌 **total_connectors**	Total hardware ports available
🚦 **is_peak_hours**	Binary flag (8–10 AM & 5–8 PM)

⚠️ ACADEMIC DISCLOSURE & MODEL DESIGN RATIONALE

- Model Type: scikit-learn DecisionTreeClassifier (max_depth=4 to prevent overfitting)
- Training Set: 600 synthetic profiles generated with random_state=42 (75% train / 25% test)
- Benchmark Accuracy: 91.3% test accuracy on synthetic demo distributions
- Ethical Disclosure: Evaluated purely on synthetic patterns. Real production deployments require real hardware telemetry. The model acts purely as an advisory reader and never writes to MongoDB.

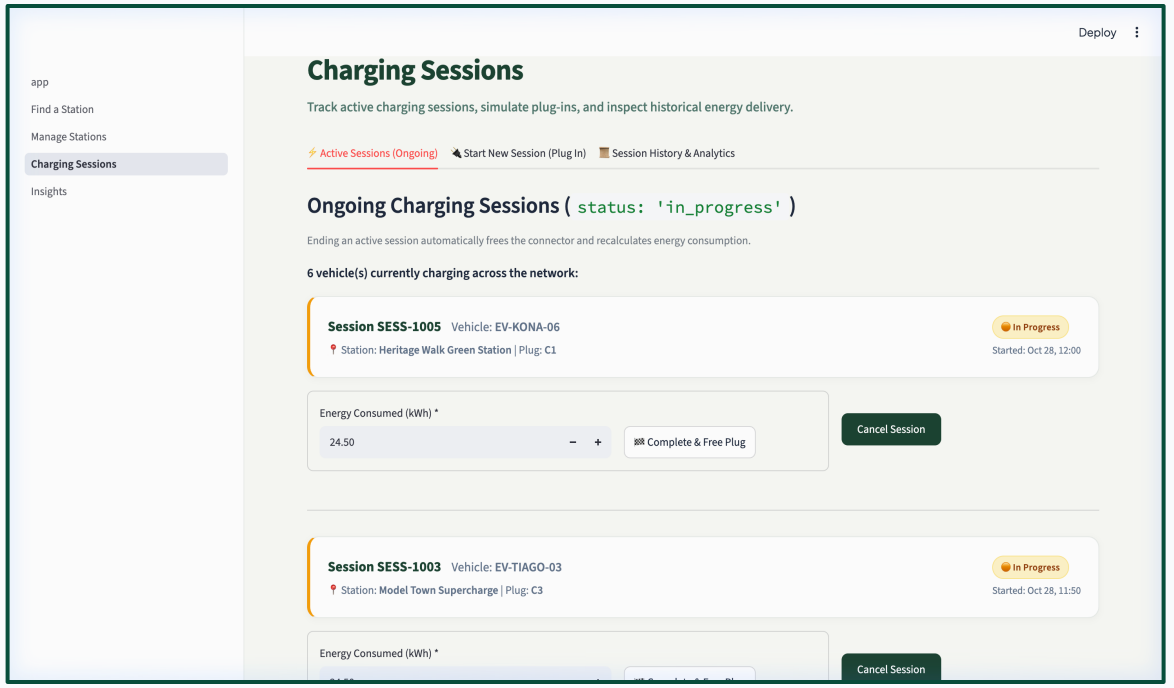
QUALITY ASSURANCE & VERIFICATION

Bulletproof Reliability: 14 Automated Tests & Index Optimization

Rigorous test suites confirming schema rejection, state machines, and compound index performance

✅ 14/14 AUTOMATED SUITE PASSES CONFIRMED

TC-01	$jsonSchema rejects negative speed (-50 kW)	PASSED
TC-02	$jsonSchema rejects invalid status ('broken')	PASSED
TC-03	Create station CS999 in MongoDB	PASSED
TC-04	Read station CS999 with regex query	PASSED
TC-05	Update station speed to 120 kW via $set	PASSED
TC-06	Start session: locks connector to 'occupied'	PASSED
TC-07	Referential Guard: Delete blocked with active session	PASSED
TC-08	Complete session: restores connector to 'available'	PASSED
TC-09	Delete station CS999 cleanly with 0 sessions	PASSED
TC-10–14	All 5 Aggregation Pipelines return valid cursors	PASSED



⚡ QUERY PERFORMANCE VERIFICATION: explain('executionStats')

```
{
  "stage": "FETCH",
  "inputStage": {
    "stage": "IXSCAN",
    "indexName": "station_id_1",
    "isMultiKey": false,
    "executionTimeMillis": 0.32,  // Sub-millisecond response!
    "totalDocsExamined": 1, "nReturned": 1
  }
}
```

🚀 PROJECT CONCLUSION & VIVA READINESS

The Road Ahead: Building Towards Smarter, Greener Cities

Summary of project achievements, key performance indicators, and future production roadmap



10	33	42	91.3%
Seeded Stations	Active Connectors	Tracked Sessions	ML Accuracy

🔮 FUTURE PRODUCTION ROADMAP

📡 Real Hardware Integration

Connect directly via OCPP 2.0.1 (Open Charge Point Protocol) via WebSockets.

🗺️ Geospatial Radius Search

Leverage MongoDB $geoNear and 2dsphere indexes for GPS proximity routing.

📱 Driver Mobile Experience

Push alerts when a high-speed plug opens up at a favorited station.

⚡ Dynamic Grid Balancing

Time-of-use tariff adjustments matching solar panel energy production spikes.

🎓 PROJECT CONCLUSION & CSE494 VIVA SUMMARY

ChargeSpot successfully demonstrates that intelligent NoSQL document modeling, atomic state transitions, aggregation pipelines, and machine learning can be combined to solve real transportation challenges.

Thank you! I am ready to answer your questions.